

Engineering LLM Applications

Build a reliable application boundary around model providers, frameworks, tools, and traces.



WHY THIS MATTERS

Start from what you already know

HTTP, JSON, schemas, LLM messages, structured output, and tool calling

- 01 Explain provider APIs in plain language.
- 02 Use streaming and retries in a small worked example.
- 03 Complete the guided lab and verify Model Context Protocol.

CORE CONCEPT

The vocabulary of this topic

Build a reliable application boundary around model providers, frameworks, tools, and traces.

provider APIs: Provider APIs expose model inference through differing names, message formats, capabilities, limits, and usage metadata.

streaming and retries: Streaming returns partial events; retries repeat only transient failures under explicit count and time limits.

LangChain components: LangChain components connect model calls, prompts, retrieval, tools, parsers, and traces behind explicit interfaces.

Model Context Protocol: MCP standardizes how applications expose scoped tools and resources to model clients.

FOUR CONNECTED IDEAS

How the concept works

01

provider APIs

02

**streaming and
retries**

03

**LangChain
components**

04

**Model Context
Protocol**

Application → provider adapter → model; tools remain application-controlled; traces record prompts, calls, latency, and usage.

CORE CONCEPT

Read every part of the mechanism

Application → provider adapter → model; tools remain application-controlled; traces record prompts, calls, latency, and usage.

provider: A hosted or local inference service.

stream: Incremental response events.

MCP: A standard protocol for model-facing capabilities.

WORKED EXAMPLE

Worked example by hand

Call two providers through one interface, require the same schema, and fall back only on retryable errors.

01

Known values

02

Apply the mechanism

03

Interpret the result

RUN IT AND INSPECT THE RESULT

Map the concept to Python

```
response = completion(  
    model="ollama/llama3.2",  
    messages=[{"role": "user", "content": "Return one JSON status"}],  
)  
print(response.choices[0].message.content)
```

OUTPUT

```
{"status": "ready"}
```

Each line corresponds to a concept already explained.

QUALITY GATE

Build the complete mental model

- ☒ **Normalize provider differences behind an application-owned interface and capability checks.**

- ☒ **Retry only transient errors with exponential backoff, jitter, and a total attempt limit.**

- ☒ **Trace model, prompt version, tool calls, tokens, latency, errors, and cost for every request.**

- ☒ **MCP tools perform actions while resources expose context; both require least-privilege access.**

FOUR CONNECTED IDEAS

Notebook readiness map

01

provider APIs

02

**streaming and
retries**

03

**LangChain
components**

04

**Model Context
Protocol**

Use the concepts from this lesson to read and complete
<https://github.com/curiously/Al-Bootcamp/blob/master/26.multiple-llm-providers-with-litellm.ipynb>.

RISK REVIEW

Common beginner mistakes

01 Retrying every error and multiplying cost.

02 Granting an MCP server broader filesystem or network access than the task requires.

GUIDED LAB

Build one traced application that streams output, validates a response, and invokes a scoped external capability.

01

02

03

04

Predict

Run

Inspect

Explain

SESSION SYNTHESIS

Keep the system, not isolated definitions.

- 01 Explain provider APIs in plain language.
- 02 Use streaming and retries in a small worked example.
- 03 Complete the guided lab and verify Model Context Protocol.

NEXT EVIDENCE

An LLM service with provider configuration, structured output, tool boundaries, traces, retry limits, and usage reporting.